

Sesión 10: Custom Painting & Canvas (Creación de Interfaces Avanzadas con CustomPainter, Canvas y Path)

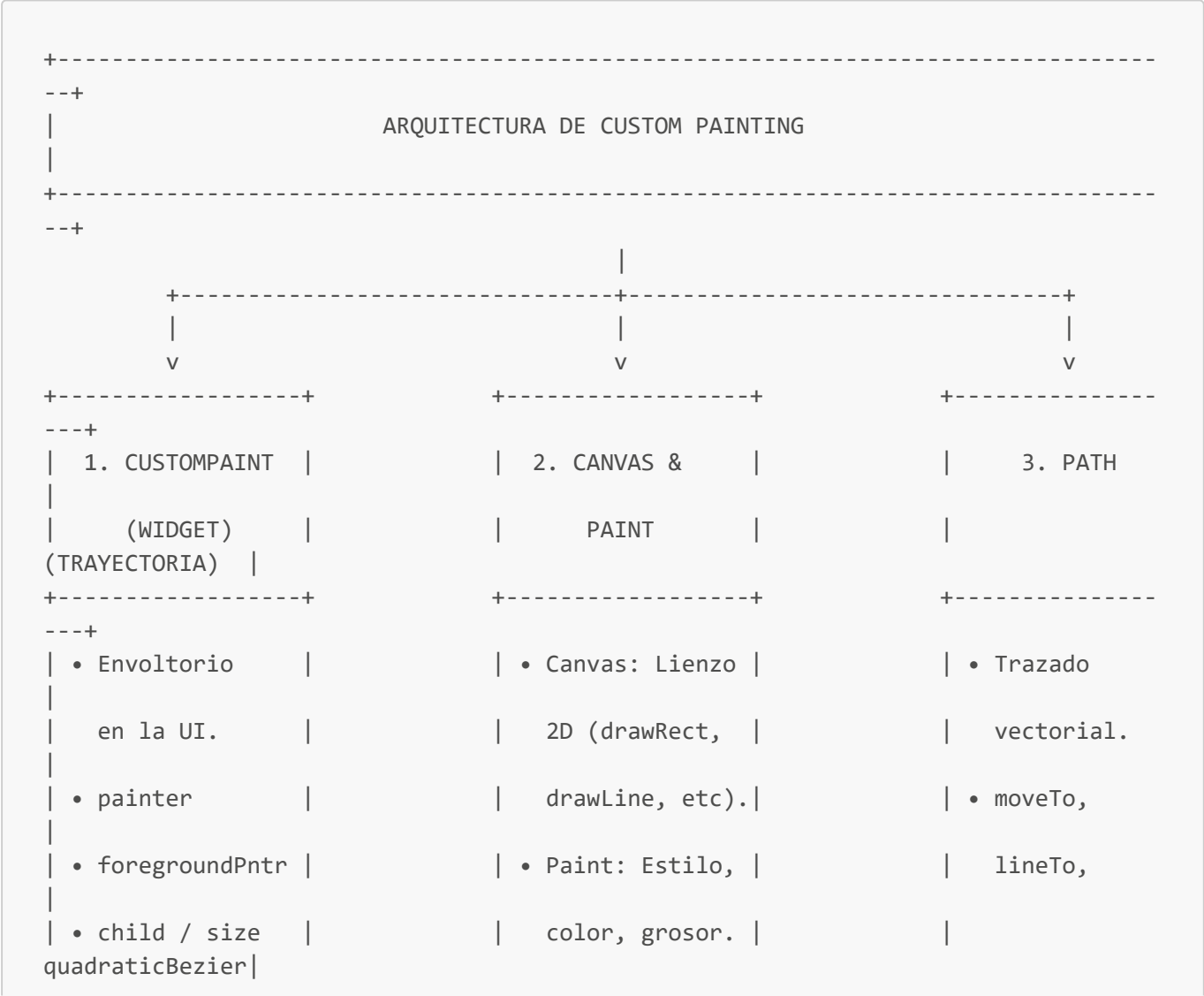
Introducción

Cuando los widgets predeterminados de maquetación y diseño no satisfacen requerimientos gráficos complejos —como dashboards con analítica vectorial, trazado de formas geométricas personalizadas, ondas de fondo o gráficos estadísticos a medida—, Flutter permite tomar control directo sobre la etapa de pintado (*paint phase*) del motor Skia/Impeller.

En esta sesión profundizaremos en la API de dibujo de bajo nivel en Flutter mediante `CustomPaint`, la clase abstracta `CustomPainter`, el lienzo de renderizado `Canvas` y la definición de trayectorias complejas con `Path`.

Conceptos y Definiciones

El dibujado personalizado trasciende el árbol de widgets convencional para interactuar directamente con el sistema de renderizado.



```
+-----+
+-----+
+-----+
---+
```

1. El Widget `CustomPaint` y la Clase `CustomPainter`

- **`CustomPaint`**: Widget que actúa como puente entre la jerarquía de widgets y la capa de dibujo bajo nivel.
- **`painter`**: Recibe una instancia de `CustomPainter` para pintar **detrás** del widget hijo.
- **`foregroundPainter`**: Pinta **por encima** del widget hijo.
- **`child`**: Widget sobre o debajo del cual se ejecutará el renderizado gráfico.
- **`size`**: Define las dimensiones del área de lienzo si no hay un `child` que las determine.
- **`CustomPainter`**: Clase abstracta que exige la implementación de dos métodos esenciales:
 1. **`paint(Canvas canvas, Size size)`**: Se invoca cada vez que el widget debe re-dibujarse. Otorga acceso directo al lienzo `Canvas` y al tamaño disponible `Size`.
 2. **`shouldRepaint(covariant CustomPainter oldDelegate)`**: Determina si el motor debe volver a ejecutar `paint()` cuando cambia el estado o las propiedades del widget, previniendo re-renderizados innecesarios y optimizando el rendimiento.

2. Elementos Principales de Dibujo: `Canvas` y `Paint`

- **`Canvas`**: Entorno gráfico bidimensional donde se ejecutan las primitivas de dibujo.
- Métodos comunes: `drawLine()`, `drawRect()`, `drawRRect()`, `drawCircle()`, `drawArc()`, `drawPath()`.
- **`Paint`**: Configura el estilo con el que se aplicarán los trazos en el `Canvas`.
- **`color`**: Color del trazo o relleno.
- **`style`**: Define si el elemento se dibuja relleno (`PaintingStyle.fill`) o solo como contorno (`PaintingStyle.stroke`).
- **`strokeWidth`**: Grosor de la línea en píxeles lógicos.
- **`strokeCap`**: Acabado de los extremos de las líneas (`StrokeCap.round`, `StrokeCap.square`, `StrokeCap.butt`).
- **`shader`**: Aplica gradientes de color (`LinearGradient`, `RadialGradient`).

3. Trazado Vectorial Complejo con `Path`

La clase `Path` permite describir formas geométricas complejas o compuestas mediante una secuencia de segmentos de recta, arcos y curvas paramétricas:

- **moveTo(x, y)**: Mueve el puntero de dibujo a la coordenada cartesiana especificada sin trazar una línea.
- **lineTo(x, y)**: Dibuja un segmento de recta desde la posición actual hasta el punto (x, y) .
- **quadraticBezierTo(x1, y1, x2, y2)**: Dibuja una curva de Bézier cuadrática hacia el punto final $(x2, y2)$ utilizando $(x1, y1)$ como punto de control de la curvatura.
- **cubicTo(x1, y1, x2, y2, x3, y3)**: Dibuja una curva de Bézier cúbica con dos puntos de control.
- **close()**: Traza una recta final desde el punto actual hasta el punto inicial de la trayectoria para cerrar el polígono.

Ejemplo Completo con Código Fuente: Tarjeta de Métricas con Fondo de Onda Vectorial y Indicador Circular

El siguiente código implementa una tarjeta de supervisión que dibuja un fondo de onda suave usando curvas de Bézier cuadráticas (**Path**) y un indicador de progreso circular totalmente personalizado mediante **CustomPainter**.

Código Fuente Explicado (**lib/main.dart**)

```
import 'dart:math' as math;
import 'package:flutter/material.dart';

void main() {
  runApp(const DemoCustomPaintingApp());
}

/// Widget Raíz
class DemoCustomPaintingApp extends StatelessWidget {
  const DemoCustomPaintingApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'ESCOM - Custom Painting',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color(0xFF6B112D), // Guinda Institucional
        ),
      ),
      home: const PantallaMonitorGrafico(),
    );
  }
}

/// Pantalla Principal con Lienzos Personalizados
class PantallaMonitorGrafico extends StatefulWidget {
  const PantallaMonitorGrafico({super.key});

  @override
```

```

State<PantallaMonitorGrafico> createState() =>
  _PantallaMonitorGraficoState();
}

class _PantallaMonitorGraficoState extends State<PantallaMonitorGrafico> {
  double _porcentajeCarga = 0.75;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Custom Painting & Canvas'),
        backgroundColor: Theme.of(context).colorScheme.primary,
        foregroundColor: Colors.white,
      ),
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(24.0),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.center,
          children: [
            const Text(
              '1. Fondo Vectorial Personalizado con Path (Curvas Bézier)',
              style: TextStyle(fontSize: 14, fontWeight: FontWeight.bold),
            ),
            const SizedBox(height: 12),

            // CONTENEDOR CON FONDO PINTADO MEDIANTE CUSTOMPAINT
            ClipRRect(
              borderRadius: BorderRadius.circular(16),
              child: CustomPaint(
                painter: DibujadorOndaFondo(),
                child: SizedBox(
                  width: double.infinity,
                  height: 160,
                  child: Padding(
                    padding: const EdgeInsets.all(20.0),
                    child: Column(
                      crossAxisAlignment: CrossAxisAlignment.start,
                      children: const [
                        Text(
                          'Análisis de Rendimiento',
                          style: TextStyle(
                            color: Colors.white,
                            fontSize: 18,
                            fontWeight: FontWeight.bold,
                          ),
                        ),
                        SizedBox(height: 4),
                        Text(
                          'Trazado dinámico de bajo nivel',
                          style: TextStyle(color: Colors.white70, fontSize: 12),
                        ),
                      ],
                    ),
                  ),
                ),
              ),
            ),
          ],
        ),
      ),
    );
  }
}

```

```

        ),
      ),
    ),
  ),
  const SizedBox(height: 32),

  const Text(
    '2. Indicador Circular Gráfico con Canvas & Paint',
    style: TextStyle(fontSize: 14, fontWeight: FontWeight.bold),
  ),
  const SizedBox(height: 20),

  // INDICADOR CIRCULAR CON CANVAS
  SizedBox(
    width: 180,
    height: 180,
    child: CustomPaint(
      painter: DibujadorIndicadorCircular(
        porcentaje: _porcentajeCarga,
        colorPrincipal: Theme.of(context).colorScheme.primary,
      ),
      child: Center(
        child: Text(
          '${(_porcentajeCarga * 100).toInt()}%',
          style: const TextStyle(
            fontSize: 28,
            fontWeight: FontWeight.bold,
          ),
        ),
      ),
    ),
  ),
  const SizedBox(height: 24),

  // Controles interactivos para redibujar
  Slider(
    value: _porcentajeCarga,
    min: 0.0,
    max: 1.0,
    onChanged: (valor) {
      setState(() {
        _porcentajeCarga = valor;
      });
    },
  ),
  Text(
    'Nivel de Carga: ${(_porcentajeCarga * 100).toStringAsFixed(0)}%',
    style: const TextStyle(color: Colors.grey),
  ),
],
),
);
}

```

```
}

/// CustomPainter para dibujar una onda suave con gradiente en el fondo
class DibujadorOndaFondo extends CustomPainter {
  @override
  void paint(Canvas canvas, Size size) {
    // 1. Definición del pincel con gradiente
    final Paint paint = Paint()
      ..shader = const LinearGradient(
        colors: [Color(0xFF6B112D), Color(0xFFB82E55)],
        begin: Alignment.topLeft,
        end: Alignment.bottomRight,
      ).createShader(Rect.fromLTWH(0, 0, size.width, size.height))
      ..style = PaintingStyle.fill;

    // 2. Definición del trazado vectorial con Path
    final Path path = Path();
    path.lineTo(0, size.height * 0.7); // Bajar por el borde izquierdo

    // Trazado de la onda usando curva Bézier cuadrática
    final puntoControl = Offset(size.width * 0.5, size.height * 1.1);
    final puntoFinal = Offset(size.width, size.height * 0.75);

    path.quadraticBezierTo(
      puntoControl.dx,
      puntoControl.dy,
      puntoFinal.dx,
      puntoFinal.dy,
    );

    path.lineTo(size.width, 0); // Subir por el borde derecho
    path.close(); // Cerrar figura en la esquina superior izquierda

    // 3. Renderizado en el lienzo
    canvas.drawPath(path, paint);
  }

  @override
  bool shouldRepaint(covariant CustomPainter oldDelegate) => false;
}

/// CustomPainter para renderizar un medidor circular
class DibujadorIndicadorCircular extends CustomPainter {
  final double porcentaje;
  final Color colorPrincipal;

  DibujadorIndicadorCircular({
    required this.porcentaje,
    required this.colorPrincipal,
  });

  @override
  void paint(Canvas canvas, Size size) {
    final Offset centro = Offset(size.width / 2, size.height / 2);
```

```

final double radio = math.min(size.width / 2, size.height / 2) - 12;

// Pincel para la pista de fondo (Anillo base)
final Paint pincelFondo = Paint()
    ..color = Colors.grey.shade200
    ..style = PaintingStyle.stroke
    ..strokeWidth = 14;

// Pincel para el arco de progreso
final Paint pincelProgreso = Paint()
    ..color = colorPrincipal
    ..style = PaintingStyle.stroke
    ..strokeWidth = 14
    ..strokeCap = StrokeCap.round;

// Dibujar pista de fondo
canvas.drawCircle(centro, radio, pincelFondo);

// Dibujar arco de progreso basado en el porcentaje
// -math.pi / 2 inicia el trazo en la parte superior del círculo (12 horas)
final double anguloBarrido = 2 * math.pi * porcentaje;

canvas.drawArc(
    Rect.fromCircle(center: centro, radius: radio),
    -math.pi / 2,
    anguloBarrido,
    false,
    pincelProgreso,
);
}

@override
bool shouldRepaint(covariant DibujadorIndicadorCircular oldDelegate) {
    // Solo vuelve a pintar si el porcentaje o el color cambian
    return oldDelegate.porcentaje != porcentaje ||
        oldDelegate.colorPrincipal != colorPrincipal;
}
}

```

Prácticas Guiadas / Laboratorio

Práctica 1: Creación de un Gráfico de Líneas Estadístico

1. Crea una clase que herede de `CustomPainter` llamada `DibujadorGraficoLineas`.
2. Pasa una lista de valores numéricos como parámetro (ej. `[10, 45, 20, 80, 60]`).
3. En el método `paint`, calcula la distancia entre puntos a lo largo del ancho del lienzo (`size.width`).
4. Utiliza un `Path` y un bucle `for` con la función `lineTo()` para conectar las coordenadas de cada dato estadístico.

5. Usa `canvas.drawPath()` asignando un `Paint` con `style = PaintingStyle.stroke` y `strokeWidth = 3` para dibujar la gráfica de líneas.
-

Práctica 2: Dibujo de Polígonos Regulares Personalizados (Estrella o Pentágono)

1. Implementa un `CustomPainter` que calcule matemáticamente los vértices de un polígono de N lados utilizando las funciones `math.cos()` y `math.sin()`.
 2. Mueve el puntero a la coordenada del primer vértice mediante `path.moveTo()`.
 3. Traza las aristas conectando los vértices subsecuentes con `path.lineTo()`.
 4. Finaliza el dibujo con `path.close()` y aplícale un relleno de gradiente radial (`RadialGradient`) mediante `Paint.shader`.
-

Resumen y Puntos Clave

- **Capa de Dibujo Bajo Nivel:** `CustomPaint` permite tomar el control de la fase de renderizado gráfico de Flutter sin las restricciones de la maquetación basada en cajas tradicionales.
 - **`CustomPainter`:** Define la lógica de dibujo dentro de `paint(Canvas, Size)` y gestiona la re-ejecución optimizada a través de `shouldRepaint()`.
 - **Primitivas en `Canvas`:** Permite renderizar líneas, círculos, rectángulos, arcos y trayectorias complejas combinados con la configuración estática o de degradados de `Paint`.
 - **Trazo Vectorial con `Path`:** Herramienta principal para maquetar geometrías no estándar mediante rectas (`lineTo`), desplazamientos (`moveTo`) y curvaturas paramétricas (`quadraticBezierTo`, `cubicTo`).
 - **Optimización de Renderizado:** Es fundamental devolver únicamente `true` en `shouldRepaint()` cuando las propiedades de las que depende el trazado hayan variado.
-

Bibliografía y Referencias

1. **Google LLC.** (2024). *CustomPainter class documentation - Flutter API*. Recuperado de: <https://api.flutter.dev/flutter/widgets/CustomPainter-class.html>
2. **Google LLC.** (2024). *Canvas class documentation - Flutter API*. Recuperado de: <https://api.flutter.dev/flutter/graphics/Canvas-class.html>
3. **Flutter Engineering Team.** (2023). *Custom Painting in Flutter: Drawing custom shapes and paths*. Recuperado de: <https://docs.flutter.dev/ui/graphics/custom-painting>
4. **Windmill, E.** (2020). *Flutter in Action* (Capítulo 10: Canvas, Paths, and Custom Painting). Manning Publications. ISBN: 978-1617296147.